

Loss in Machine Learning
 metric for quantifying performance
 Optimization algorithm is trying to minimize or maximize this function.

1) Mean Squared Error

$$J = \frac{1}{14} \sum_{i=1}^n (\hat{y}_i - y_i^{true})^2$$

$$\hat{w} = \min_w J \Rightarrow \text{Gradient Descent}$$

 weights of the model
Adar
Regressions

2) Binary Cross Entropy Loss

$$L = - \sum_{i=1}^K p_i \log p_i =$$

 $K \Rightarrow \# \text{ of classes}$
 $K = 2$

$$= - \left(\underbrace{P(Y=0)}_{\text{true label}} \log \underbrace{P(\hat{Y}=0)}_{\text{predicted label}} + P(Y=1) \log P(\hat{Y}=1) \right)$$

If true label = 0

$$Y = \{0, 1\}$$

During training

$$J = - \log P(\hat{Y}=0)$$

During training w will be chosen
so that $P(\hat{Y}=0)$ approaches 1

If true label = 1

During training

$$J = - \log P(\hat{Y}=1)$$

w will be chosen so
that $P(\hat{Y}=1)$ approaches 1

3) Multi-Class Cross Entropy

$$J = \sum_{i=1}^M \sum_{j=1}^K I_i^j \log P(\hat{Y}_i = j)$$

$M \Rightarrow$ total training examples

$K \Rightarrow$ total classes.

I_i^j indicator

$I^j \Rightarrow$ true function

$$I^j = \begin{cases} 1 & \text{if true class} = j \\ 0 & \text{otherwise} \end{cases}$$

During training:

Assume true class ~~is~~ j^* is known

$$J = -\log P(Y = j^*)$$

LeNet Parameter Calculation

- + To understand how weights are increasing in model
- + which layers are taking up more weights

Parameters Calculation

Layer	Type	Input Shape	Output Shape	Parameters
Input	Image	$32 \times 32 \times 1$	$32 \times 32 \times 1$	0
C1	Conv (5×5) Stride=1	$32 \times 32 \times 1$	$28 \times 28 \times 6$	# of weights in kernels + # of bias term $= 6 \times (5 \times 5) + 6$

S2

Augpool (2x2) 28x28x6

$14 \times 14 \times 6$

$$= 156$$

C3

Conv(Sx5) 14x14x6

10x10x16

$$= 16 \times 6 \times (5 \times 5) + 16$$
$$= 2416$$

S4

Aug Pool (2x2) 10x10x16

ΣxΣx | 6

— flattering eyes

FS

feed forward 400

120

$$= 400 \times 120 \div 100$$

$$= 48,000$$

FG

feed forward 120

84

$$= 120 \times 84 + 84$$

$$= 10,164$$

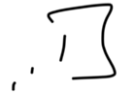
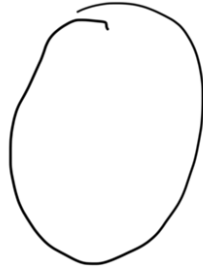
F7

feed forward 24

10

11 84x10 f 10
11 850

Total = 61,706



16 features

bias = 16

kernel = 2

Every output feature map should have contribution from each input feature map

1 output feature map = one kernel applied to input feature map 1
 require 6 kernels

+
 one kernel applied to input feature map 2
 +

+
 one kernel applied to input feature map 6
 +

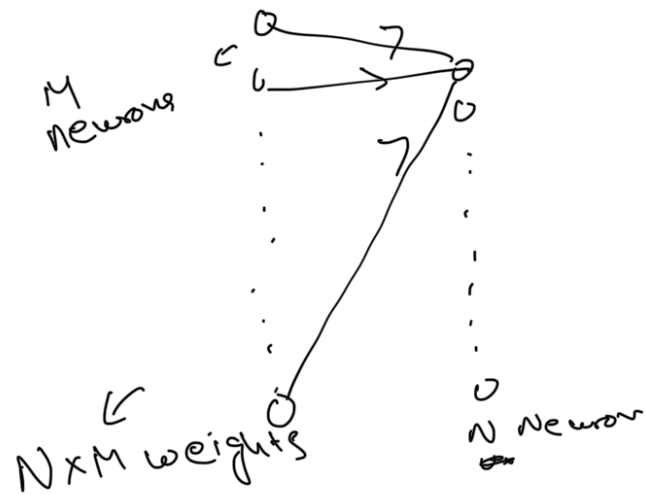
1 6
 16 2
 $x = 16 \times 6$

Number of kernels required for a convolutional layer = # of output feature map $\times$ # of input feature map

+
 b

Total number of weights = $\# \text{ of output feature maps} \times \# \text{ of input features (SXS)}$
 $+ \# \text{ of output feature maps}$

For feed forward network



$$x = \underline{\underline{N \times M}}$$